

AI AGENT HUNTING

Tác tử điều tra mối đe dọa bị ràng buộc bởi bằng chứng,
vận hành theo khung PEAK của Splunk

Báo cáo nghiên cứu kỹ thuật

Hệ thống: gói Python hunting v0.1.0

Khung quy trình: PEAK — Prepare, Execute, Act + Knowledge

Dữ liệu đánh giá: BOTS v1 (eval 4.42M dòng + sample 16 sự kiện)

Kết quả kiểm thử: 514 passed, 13 skipped, 0 failed

Ngày lập báo cáo: 2026-09-22

MỤC LỤC

(Nhấn Ctrl+A rồi F9 trong Word để cập nhật mục lục và số trang.)

Chương 1 — Mở đầu

Chương này đặt bối cảnh, phát biểu bài toán, nêu câu hỏi nghiên cứu, liệt kê đóng góp và khoanh vùng phạm vi. Các chương sau đi vào cơ sở lý thuyết (Chương 2), thiết kế (Chương 3–5), thực nghiệm (Chương 6), kết quả (Chương 7), và bàn luận/kết luận (Chương 8–9).

1.1. Bối cảnh

Săn mồi đe dọa (threat hunting) là quy trình chủ động đi tìm những sự cố an ninh mà các cơ chế phát hiện tự động — luật (rule), chữ ký (signature), cảnh báo (alert) — đã bỏ sót. Khác với phòng thủ thụ động chờ cảnh báo bật lên, người săn mồi đe dọa xuất phát từ một giả định: kẻ địch **có thể đã ở trong mạng**, và nhiệm vụ là tìm bằng chứng khẳng định hoặc bác bỏ giả định đó. Đặc trưng phương pháp luận quan trọng nhất của săn mồi đe dọa là: một cuộc săn không tìm thấy gì vẫn có giá trị, miễn là nó ghi lại rõ đã tìm ở đâu và chưa tìm ở đâu — tức là phân biệt được "không có bằng chứng về tấn công" với "có bằng chứng về sự vắng mặt của tấn công".

Trong vài năm gần đây, mô hình ngôn ngữ lớn (LLM) được kỳ vọng tự động hóa săn mồi đe dọa: đọc một câu hỏi tiếng người, sinh truy vấn cho SIEM, đọc kết quả, rồi kết luận. Kỳ vọng này hấp dẫn nhưng nguy hiểm, vì bản chất của LLM là sinh văn bản trôi chảy chứ không phải bảo đảm tính đúng đắn của bằng chứng. Một tác tử điều tra để LLM tự do để tạo ra kết luận nghe hợp lý nhưng không có cơ sở quan sát.

1.2. Phát biểu bài toán

Một nhà phân tích thường bắt đầu từ một trong các dạng đầu vào sau:

- Một câu hỏi tự do: *"Amber Turing đã vào một trang đối thủ. Tên miền đó là gì?"*
- Một giả thuyết: *"Máy chủ web này bị khai thác qua thành phần tìm kiếm Joomla."*
- Một định danh chuẩn: một mã CVE (CVE-2024-21887), một kỹ thuật MITRE ATT&CK (T1059.001), một chỉ báo xâm phạm (IOC).
- Một cảnh báo EDR chỉ có tên máy chủ và thời điểm.

Câu trả lời đúng phụ thuộc vào telemetry thật — Splunk, cơ sở SQLite kiểm thử, và về sau có thể là EDR hoặc thư điện tử. Ba khó khăn cốt lõi:

1. **Schema không thống nhất, và tên nguồn không phải nghĩa của dữ liệu.** Một nguồn được đặt tên "SMTP" nhưng thiếu trường người gửi và người nhận thì không phải là nguồn thư. Một tên máy chứa chuỗi `srv` không chứng minh đó là máy chủ.
2. **Truy vấn rộng có nhiều nghĩa.** Rộng có thể vì tấn công không xảy ra, hoặc vì nguồn không phủ, hoặc vì truy vấn bị cắt cụt, hoặc vì nhà cung cấp không tới được.
3. **Từ khóa dễ dẫn tới thiên kiến kịch bản.** Nếu hệ thống thấy chữ "email" rồi tự động chạy một kịch bản điều tra email, nó đã áp một câu chuyện lên dữ liệu thay vì để dữ liệu nói.

Từ đó, ba lỗi cố hữu mà hệ thống này được xây để **chặn bằng hợp đồng, không bằng lời nhắc**:

L1. Mô hình ngôn ngữ viết thẳng truy vấn gốc (SPL/SQL/KQL) rồi tự tuyên bố kết quả.

L2. Một từ khóa trong câu hỏi (email, Tor, CVE, web) chọn sẵn một kịch bản điều tra cố định.

L3. Một truy vấn trả về rỗng và hoàn tất bị đọc thành "không có tấn công" (kết luận lành tính).

1.3. Câu hỏi nghiên cứu

- **CH1.** Có thể xây một tác tử sẵn mối đe dọa dùng LLM mà quyết định về sự thật, truy vấn và điểm dừng thuộc về logic tất định, còn LLM chỉ đóng vai đề xuất, hay không?
- **CH2.** Có thể ánh xạ một quy trình sẵn được ngành công nhận (PEAK của Splunk) lên một hệ thống phần mềm mà không đánh mất bản chất "PEAK là quy trình cho con người", đồng thời làm cho bước Prepare trở thành ràng buộc cứng, hay không?
- **CH3.** Trên dữ liệu tấn công thật, hệ thống có đạt độ đặc hiệu cao (ít dương tính giả) mà vẫn bắt được tấn công thật (có độ nhạy) hay không, và đo được đến đâu?

1.4. Đóng góp

Báo cáo này trình bày các đóng góp sau:

4. **Kiến trúc ba đồ thị có ranh giới nhận thức chặt** (Chương 4): tách SemanticGoalGraph (mục tiêu), CapabilityGraph (năng lực nguồn), EvidenceGraph (bằng chứng đã kiểm), với luật bất biến rằng chỉ bộ kiểm tất định mới nâng một cạnh từ "chưa chứng minh" lên "đã chứng minh".
5. **Hiện thực hóa bốn cửa của khung PEAK** (Chương 3): Prepare, Execute/Refine, Act, và M-ATH — trong đó M-ATH thay bước "train mô hình cục bộ" bằng gọi một mô hình pretrained bên ngoài, có cơ chế grounding.
6. **Cổng PEAK Prepare bắt buộc trên mọi đường sẵn** (Chương 5, đóng góp mới nhất của phiên bản này): trước đây Prepare chỉ bắt buộc trên đường PoC; nay nó là ràng buộc cứng cho cả đường giả thuyết, baseline và M-ATH, với cơ chế suy plan tự động và van thoát tường minh.
7. **Đánh giá thực nghiệm trên BOTS v1** (Chương 6–7): đo độ đặc hiệu trên 4.38 triệu dòng lệnh tính và độ nhạy trên tấn công Joomla thật, kèm phân tích trung thực về những gì chưa đo được.

1.5. Phạm vi và giới hạn phạm vi

Trong phạm vi: telemetry doanh nghiệp qua Splunk và một backend SQLite kiểm thử (gọi là CDB); các loại đầu vào giả thuyết/CVE/TTP/IOC/cảnh báo/PoC; đánh giá trên BOTS v1.

Ngoài phạm vi hiện tại: EDR, IDS, thư điện tử như nguồn thật (chỉ trở thành nguồn khi có adapter và kiểm thử năng lực tương ứng); chứng nhận PEAK chính thức của Splunk (kho hiện thực hóa quy trình, không nhận chứng nhận); và mọi tuyên bố khái quát ra "mọi SIEM" — một lần chạy lại chỉ chứng minh đúng hành vi đã đo của lần đó.

1.6. Cấu trúc báo cáo

Chương 2 trình bày cơ sở lý thuyết và công trình liên quan. Chương 3 mô tả khung PEAK và cách hệ thống hiện thực hóa. Chương 4 trình bày kiến trúc và mô hình nhận thức năm tầng. Chương 5 đặc tả đóng góp mới — cổng PEAK Prepare bắt buộc. Chương 6 mô tả phương pháp thực nghiệm (môi trường, dữ liệu, thiết kế test). Chương 7 báo cáo kết quả và tỉ lệ. Chương 8 bàn luận và nêu hạn chế. Chương 9 kết luận và nêu hướng phát triển. Cuối cùng là tài liệu tham khảo và phụ lục.

Chương 2 — Cơ sở lý thuyết và công trình liên quan

Chương này tách nguyên tắc được tài liệu bên ngoài nâng đỡ khỏi các lựa chọn kỹ thuật cục bộ. Mã định danh REF- khớp bảng truy vết trong 03_LITERATURE-AND-TRACEABILITY.md; đường dẫn đầy đủ ở Chương 10. Lưu ý phương pháp luận: "nâng đỡ" nghĩa là ủng hộ một *nguyên tắc*, không phải "chứng minh đúng một hiện thực cụ thể".*

2.1. Định nghĩa và bản chất của sẵn mỗi đe dọa

Sẵn mỗi đe dọa là quá trình thủ công hoặc có máy hỗ trợ nhằm tìm những sự cố mà cơ chế phát hiện tự động bỏ sót. Điểm phân biệt với giám sát thụ động là **giả định vi phạm** (assume breach): người sẵn không chờ cảnh báo mà chủ động kiểm một giả thuyết về hoạt động của kẻ địch. Hệ quả phương pháp luận: kết quả của một cuộc sẵn không phải nhị phân "có/không tấn công", mà là một trong nhiều trạng thái — đã khẳng định, đã bác bỏ, không kết luận vì thiếu phủ, không kết luận vì thiếu năng lực nguồn, v.v. Hệ thống trong báo cáo này giữ đúng sự phân biệt đó thay vì ép một nhãn nhị phân (xem Chương 4, mô hình nhận thức).

2.2. Điều tra đa chặng và đồ thị nguồn gốc

Một nhánh nghiên cứu lâu năm dựng lại "đồ thị nguồn gốc" (provenance graph) từ nhật ký hệ thống để truy vết một cuộc xâm nhập qua tiến trình, tệp và kết nối mạng:

- **SLEUTH** [REF-SLEUTH] (Hossain và cộng sự, USENIX Security 2017) tái dựng kịch bản tấn công thời gian thực từ dữ liệu audit COTS bằng cách lan truyền nhãn nghi ngờ trên đồ thị phụ thuộc.
- **HOLMES** [REF-HOLMES] (Milajerdi và cộng sự, IEEE S&P 2019) phát hiện APT bằng cách tương quan các luồng thông tin đáng ngờ giữa các thực thể và ánh xạ chúng lên các bước của một chuỗi tấn công.
- **OmegaLog** [REF-OMEGALOG] (Hassan và cộng sự, NDSS 2020) hòa giải ngữ cảnh ứng dụng, hệ thống và mạng để một sự kiện giữ được độ trung thực khi đi qua nhiều tầng log.

Điều các công trình này nâng đỡ: giữ một *đồ thị bằng chứng có trích dẫn* thay vì một đoạn văn kết luận của mô hình. **Điều chúng không thiết lập:** rằng mọi câu hỏi phải đi qua một đồ thị tấn công cố định. Đây là căn cứ để hệ thống cho phép một câu hỏi tra cứu thuộc tính (ví dụ hỏi tên miền, hỏi phiên bản phần mềm) dừng ở một quan hệ đã chứng minh, mà không bắt buộc dựng một chuỗi tấn công đầy đủ.

2.3. Truy vấn logic có kiểu đứng trước cú pháp gốc

Việc để lớp ngữ nghĩa có kiểu đứng trước cú pháp truy vấn gốc có tiền lệ:

- **AIQL** [REF-AIQL] (Gao và cộng sự, USENIX ATC 2018) định nghĩa một ngôn ngữ truy vấn hành vi có kiểu trên dữ liệu giám sát hệ thống, rồi lập kế hoạch thực thi hiệu quả.
- **ThreatRaptor** [REF-THREATRAPTOR] (Gao và cộng sự, ICDE 2021) tách bước rút hành vi có cấu trúc từ báo cáo tình báo khỏi bước tổng hợp truy vấn.

Hệ thống này vay chính cấu trúc đó: SemanticGoalGraph và QueryIntent là lớp trung gian có kiểu; SPL và SQL chỉ được adapter biên dịch ra ở bước cuối. Điều này trực tiếp chặn lỗi **L1** (mô hình viết thẳng truy vấn gốc): mô hình không xuất SPL, nó xuất một đồ thị mục tiêu có kiểu, và một cổng AST tối thiểu (NativeQueryGate) mới quyết định một ứng viên SPL có được chạy hay không.

2.4. Vòng giả thuyết – bằng chứng – hành động, và sự không chắc chắn tường minh

- **Evidential Cyber Threat Hunting** [REF-ECTH] (tiền ấn phẩm arXiv:2104.10319) mô hình hóa tri thức, giả thuyết và hành động cùng độ không chắc chắn tường minh.
- **ATHAFI** [REF-ATHAFI] (arXiv:2003.03663) thu thập telemetry một cách thích nghi để kiểm giả thuyết.
- **TaHiTI** [REF-TAHITI] (FI-ISAC) là vòng đời sẵn theo giả thuyết mà giới ngân hàng dùng: chuẩn bị → sẵn → kết thúc bằng phát hiện hoặc bằng khoảng trống.
- **Maxam và cộng sự** [REF-USENIX-TH] (USENIX Security 2024) đo thực địa và cho thấy quy trình sẵn rất đa dạng — một workflow duy nhất áp cho mọi câu hỏi là một giả định yếu.

Điều được nâng đỡ: việc *gọi tên* sự không chắc chắn thay vì ép một nhãn âm tính. Đây là căn cứ để hệ thống giữ nhiều quyết định dừng tách biệt (đã giải, bị bác, không kết luận vì thiếu phủ/năng lực/không tới được nguồn/hết ngân sách) và trực tiếp chặn lỗi **L3** (đọc rỗng thành lành tính). Ngưỡng ngân sách cụ thể và tên enum là lựa chọn cục bộ, không do bài báo quy định.

2.5. Schema, năng lực và khớp schema bằng mô hình có kiểm sau

Nền tảng cho khái niệm CapabilityGraph (một thao tác nhà cung cấp khai báo kiểu đầu vào, kiểu sự thật đầu ra, phân vùng, quyền, phân trang, độ hoàn tất):

- **OCSF** [REF-OCSF] — lược đồ sự kiện trung lập nhà cung cấp, mở rộng được.
- **MITRE ATT&CK Data Components** [REF-MITRE-DC] và **Detection Strategies** [REF-MITRE-ANALYTICS] — tính chất quan sát được của từng kỹ thuật và chiến lược phát hiện mức cao.
- **OpenTelemetry** [REF-OTEL] — chuẩn hóa sự kiện và thuộc tính.
- **Microsoft Threat Hunting Assistant** [REF-MICROSOFT] — trợ lý truy vấn phải biết bảng và schema, và vẫn cần người duyệt.

Về khớp schema bằng mô hình ngôn ngữ có kiểm sau: **Schema Matching with LLMs** [REF-SCHEMA-LLM] cho thấy mô hình sinh được ứng viên khớp và kích thước ngữ cảnh ảnh hưởng chất lượng; **ReMatch** [REF-REMATCH] tăng cường truy hồi trước khi khớp; **CHESS** [REF-CHESS] tách truy hồi/chọn schema/sinh truy vấn/kiểm thành các giai đoạn và cố ý giảm ngữ cảnh gửi cho mô hình; **RAT-SQL** [REF-RAT-SQL] liên kết schema theo quan hệ thay vì danh sách phẳng; **ToolShed** [REF-TOOLSHED] và **MDB-Link** [REF-MDB-LINK] bàn về cân bằng độ phủ và kích thước ngữ cảnh; **Adaptive-K** [REF-ADAPTIVE-K] được ghi là phương án đã xem và **không chọn**, vì một ngưỡng điểm có thể loại telemetry điểm thấp nhưng vẫn đúng.

Hệ thống áp các nguyên tắc này vào CapabilityBatcher và SourceProfiler: điểm liên quan chỉ sắp thứ tự xử lý (không có Top-K loại nguồn); ứng viên của mô hình chỉ trở thành năng lực thật sau khi validator đối chiếu census và adapter chạy probe có giới hạn. Điều này chặn lỗi **L2** (thiên kiến kích bản): nguồn được chọn vì hợp đồng kiểu khớp quan hệ, không vì tên nguồn trùng một từ khóa.

2.6. Dùng công cụ an toàn và đánh giá tác tử

- **Verifiably Safe Tool Use** [REF-SAFE-TOOLS] tách ý định khỏi thực thi công cụ đã kiểm.
- **Retrieve-Plan-Generation** [REF-RPG] (EMNLP 2024) lập kế hoạch lập có điều kiện theo bằng chứng đã lấy.
- **ExCyTin-Bench** [REF-EXCYTIN] (Microsoft Research) là bằng thử tác tử điều tra đa bước trên đồ thị bằng chứng.
- **DARPA Transparent Computing** [REF-DARPA-TC] cung cấp dữ liệu nguồn gốc cho đánh giá.

Các nguồn này nâng đỡ ranh giới: mô hình không gọi nhà cung cấp trực tiếp; ngân sách cuộc gọi/token/truy vấn/thời gian được ghi; độ chính xác ánh xạ nguồn, cạnh bằng chứng và câu trả lời phải đo **riêng**. Chúng không làm một mô hình trở nên đáng tin chỉ vì đã bị giới hạn.

2.7. Định vị của hệ thống so với công trình liên quan

Bảng dưới định vị hệ thống so với các nhánh trên:

Bảng 2.1.

Nhánh nghiên cứu	Kế thừa gì	Khác biệt của hệ thống này
Provenance graph (SLEUTH/HOLMES/OmegaLog)	Đồ thị bằng chứng có trích dẫn	Không ép mọi câu hỏi qua một đồ thị tấn công cố định
Typed query (AIQL/ThreatRaptor)	Lớp ngữ nghĩa có kiểu trước cú pháp gốc	Cổng AST tối thiểu kiểm ứng viên SPL do mô hình đề xuất
Evidential loop (ECTH/ATHAFI/TaHiTi)	Sự không chắc chắn tường minh	Nhiều quyết định dừng tách biệt, âm tính phải được cấp phép
Schema matching bằng LLM (CHESS/ReMatch/...)	Batch có kiểm, giảm ngữ cảnh	Không Top-K loại nguồn; probe xác nhận trước khi tin
Safe tool use (SAFE-TOOLS/RPG)	Tách ý định khỏi thực thi	Mô hình không chạm nhà cung cấp; ngân sách ghi lại

2.8. Những gì vẫn là giả thuyết kỹ thuật của kho

Theo 03_LITERATURE-AND-TRACEABILITY.md, các điểm sau **chưa** được chứng minh bên ngoài: đúng các lớp đồ thị cụ thể; công thức chấm hành động; ngưỡng ngân sách; số cuộc gọi mô hình tối ưu; ngưỡng bằng chứng tối thiểu; khả năng khái quát từ một bộ Splunk sang mọi SIEM; và mọi số F1 chưa có nhãn. Các bài bình duyệt và chuẩn chính thức nâng đỡ **nguyên tắc**; tiền ấn phẩm và tài liệu nhà cung cấp nâng đỡ **thiết kế** với trọng số yếu hơn; một lần chạy lại trong kho chỉ chứng minh đúng hành vi đã đo của lần đó.

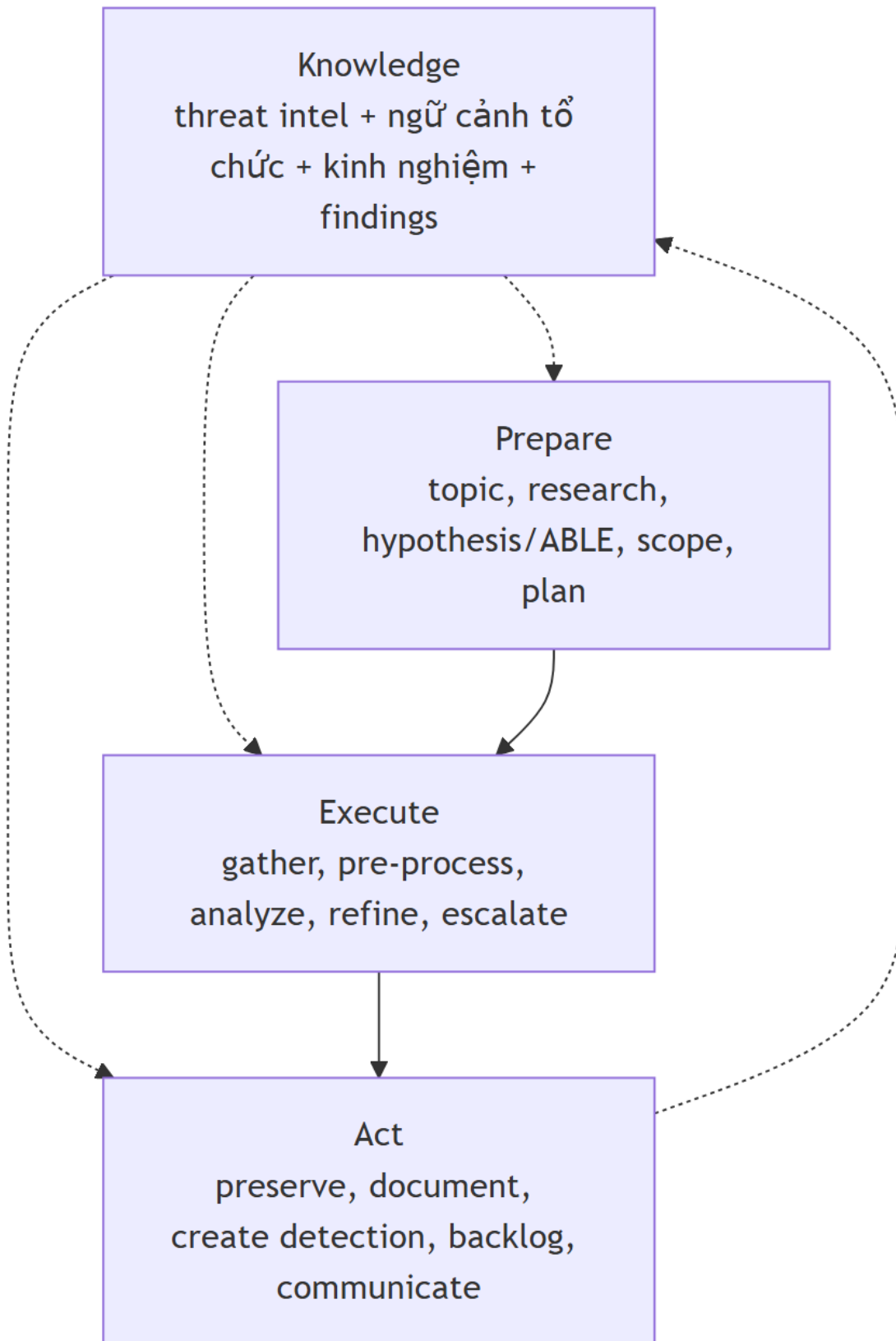
Chương 3 — Khung PEAK của Splunk và cách hệ thống hiện thực hóa

Chương này trình bày khung PEAK như một quy trình sẵn của ngành, rồi đối chiếu từng cửa của PEAK với thành phần kỹ thuật tương ứng trong kho, kèm ranh giới trung thực giữa "ngôn ngữ trình bày với SOC" và "bản chất kỹ thuật".

3.1. PEAK là gì

PEAK là khung sẵn mồi đe dọa hiện đại của Splunk SURGe, giới thiệu bởi David Bianco và Ryan Fetterman năm 2023 [REF-PEAK], thay cho các khung cũ Sqrrl (2015) và TaHiTI (2018). Tên PEAK là chữ viết tắt của ba phase cộng một yếu tố xuyên suốt:

- **Prepare** — chuẩn bị: chọn đề tài, nghiên cứu, phát biểu giả thuyết, xác định phạm vi và kế hoạch.
- **Execute** — thực thi: thu thập, tiền xử lý, phân tích, tinh chỉnh giả thuyết, leo thang phát hiện nghiêm trọng.
- **Act** — hành động: bảo tồn, tài liệu hóa, tạo detection, đưa vào backlog, truyền đạt.
- **Knowledge** — tri thức: threat intel, ngữ cảnh tổ chức, kinh nghiệm hunter, và chính findings của cuộc sẵn — thấm vào cả ba phase và được vòng lại qua Act.



Hình 3.1. Sơ đồ minh họa

Bản chất PEAK là quy trình cho nhà phân tích con người (mở Splunk, đọc log, viết detection), không phải một đặc tả kiến trúc phần mềm. Đây là điểm cần nói đúng khi báo cáo: kho này không phải là "một hiện thực PEAK được chứng nhận", mà là một hệ thống hiện thực hóa bốn cửa của quy trình đó.

3.2. Ba loại hunt của PEAK

PEAK định nghĩa ba loại hunt, cùng chạy qua ba phase Prepare → Execute → Act:

3.2.1. Hypothesis-Driven (theo giả thuyết)

Cách làm cổ điển: đặt giả thuyết về hoạt động kẻ địch rồi dùng dữ liệu xác nhận hoặc bác bỏ. PEAK khuyến nghị tạo giả thuyết tốt qua ba bước: chọn **topic** (vùng quan tâm, ví dụ "rò rỉ dữ liệu"), làm cho **testable** (ví dụ "rò rỉ qua DNS tunneling"), rồi **refine** cho huntable (ví dụ "rò rỉ dữ liệu tài chính qua DNS tunneling"). Giả thuyết không bất biến — tinh chỉnh trong lúc hunt là bình thường.

3.2.2. Baseline / EDA (khảo sát nền)

Mục đích: vẽ chân dung "bình thường" để soi lệch. Dùng khi onboard nguồn log mới, vào môi trường mới, hoặc dọn đường cho hypothesis/M-ATH. Các bước Execute đặc trưng: xây **data dictionary** (tên trường, mô tả, kiểu dữ liệu), xem **distributions** (mean/median, top values, cardinality), điều tra **outliers** (stack counting, z-score), phân tích **gaps** (nguồn thiếu, trường parse sai). PEAK khuyến nghị cửa sổ 30–90 ngày cho hầu hết nguồn.

3.2.3. M-ATH (Model-Assisted Threat Hunting)

Dùng thuật toán/mô hình để tìm lead khi phương pháp đơn giản không đủ. PEAK nêu bốn tiêu chí dùng ML (tiêu chí 1 bắt buộc): (1) phương pháp đơn giản đã thử mà không đủ chính xác; (2) có thể gán nhãn benign/malicious; (3) dữ liệu high-volume khó tóm tắt; (4) identify được nhưng classify khó → cần analyst-in-the-loop. Họ thuật toán gồm classification (ví dụ phát hiện PowerShell obfuscated), clustering, anomaly detection.

3.3. Mô hình ABLE

PEAK dùng mô hình **ABLE** để biến một giả thuyết thành kế hoạch hành động:

Bảng 3.1.

Chữ	Nghĩa	Ví dụ (DNS exfil)
Actor	Ai đánh (được phép trống)	Không rõ actor cụ thể
Behavior	TTP cụ thể, 1–2 mảnh	DNS tunneling
Location	Đánh ở đâu trong mạng	Máy + server phòng tài chính
Evidence	Cần nguồn nào, trùng thì trông thể nào	DNS logs; query lạ, record type lạ

3.4. Lập trường của kho về PEAK

Kho này **không nhận chứng nhận PEAK** của Splunk. Nó hiện thực hóa bốn cửa của quy trình PEAK và giữ nguyên những thứ PEAK không quy định: hợp đồng bằng chứng, kiểm chứng quan hệ, và trần chi phí LLM. Bốn cửa PEAK nằm trên đường PoC và (từ phiên bản này) trên mọi đường sẵn qua cổng Prepare bắt buộc; chúng không thay kiến trúc ba đồ thị.

3.5. Đối chiếu ngôn ngữ PEAK với bản chất kỹ thuật

Cách đọc: cột "Ngôn ngữ PEAK" là cách trình bày với SOC/quản lý; cột "Bản chất" là sự thật kỹ thuật để báo cáo không quá đà.

Bảng 3.2.

Ngôn ngữ PEAK	Bản chất trong hệ thống	Vị trí trong mã
Prepare: hypothesis + ABLE + scope + plan	Cổng bắt buộc từ chối chạy nếu thiếu topic, behavior, location, evidence, scope, max_duration, plan, research_refs. Actor được để trống. Token cụ thể trong ABLE (file, flag, IP, chuỗi trích dẫn, host có chữ số, DOMAIN\user) thành predicate; văn xuôi không thành predicate	hunting/peak.py, hunting/cli.py, poc/compiler.py
Execute: gather → analyze → refine → escalate	PocAgent chạy pass 1, ghi analyze, tối đa một pass refine (siết host hoặc chạy lại predicate gốc, không nối operator), rồi ghi gói IR khi có finding. Engine ghi cùng quyết định vào peak_execute_log mà không mở query không trần	poc/agent.py, engine.py, peak.py
Baseline / EDA	--baseline giữ EDA tất định (data dictionary, distribution, outlier, gap), rồi Act qua commit_act	baseline/baseline.py, act/act.py
M-ATH	--math gửi mẫu hàng tới API LLM trong .env (model pretrained, không train cục bộ); lead chỉ sống nếu giá trị nằm trong hàng đã kéo (grounding)	mathunt/mathunt.py
Act: detection + backlog + communicate	validate_spl kiểm SPL đọc-only; khi provider là Splunk, POST /services/search/parser kiểm cú pháp mà không chạy search; backlog append backlog.jsonl; stakeholder ghi Markdown	act/act.py
Knowledge	MITRE refs + research_refs + PoC library + ledger các lần chạy (baselines/, models/math_runs/,	compiler/knowledge_base.py, artifacts/

3.6. M-ATH: model pretrained ngoài thay cho train cục bộ

Điểm khác biệt quan trọng so với M-ATH kinh điển: M-ATH của PEAK giả định train/tune một mô hình ML tại chỗ. Kho này **không train model**. Cụ thể:

- --math gửi một mẫu hàng có giới hạn tới model pretrained qua API cấu hình trong .env.
- Lead do mô hình đề xuất chỉ được giữ khi giá trị của nó **xuất hiện thật trong hàng đã kéo** (grounding); lead bị loại.
- Grep toàn kho numpy|sklearn|torch|tensorflow|pandas trả về **0 kết quả** — không có mã train/fit, không có trọng số lưu cục bộ.
- Khi không có API hoặc chạy --llm stub, một prefilter thư viện chuẩn (giá trị hiếm, điểm từ vựng, chuỗi hiếm, điểm DGA) chạy và được ghi rõ heuristic_prefilter, không gọi là model đã train.

Nhãn nguồn mô hình phản ánh minh bạch: api_llm (API ground được),
api_llm_ungrounded_fallback (API trả lời nhưng không ground được → về prefilter),
heuristic_prefilter (không có API).

3.7. Ranh giới trung thực

Bốn điểm phải nói đúng khi báo cáo, tránh quá đà:

8. **ABLE chỉ lái query bằng token cụ thể.** Một câu behavior thuần văn xuôi, không có file/flag/IP/host/account, không thêm predicate.
9. **Baseline demo có thể ngắn hơn 30–90 ngày** PEAK khuyến nghị.
10. **Refine không nói `EQUALS` thành `CONTAINS`.** Pass thứ hai chỉ siết host hoặc chạy lại predicate gốc.
11. **`VALIDATED` chỉ nghĩa là parser Splunk chấp nhận cú pháp**, không phải một detection đã phát hành với tỉ lệ dương tính giả đã đo.

Các đối chiếu trong chương này khớp bảng truy vết 03_LITERATURE-AND-TRACEABILITY.md (dòng "PoC path runs Prepare, one refine, IR handoff and Act — REF-PEAK").

Chương 4 — Kiến trúc hệ thống và mô hình nhận thức

Chương này trình bày mô hình nhận thức năm tầng (nền cho mọi ranh giới), ba đường vào của hệ thống, các khối của đường sẵn chuẩn, sơ đồ triển khai, và các hợp đồng dữ liệu cốt lõi.

4.1. Mô hình nhận thức: năm tầng sự thật

Nguyên tắc trung tâm của kiến trúc: hệ thống phân biệt **năm tầng sự thật**, và trộn chúng là lỗi thiết kế chứ không chỉ lỗi diễn đạt.

Bảng 4.1.

Tầng	Vật mang (kiểu dữ liệu)	Ai được tạo	Mức độ được tin
1. Yêu cầu	HuntRequest, Alert	Người dùng	Là mục tiêu, chưa phải bằng chứng
2. Đề xuất	SemanticGoalGraph, SourceCapabilityProposal	Mô hình ngôn ngữ, trong schema	Không tin, cho đến khi bộ kiểm tất định nhận
3. Năng lực	CapabilityGraph, RuntimeCapability	Census + probe thành công	Tin ở mức "nguồn này quan sát được vai trò này", không phải "sự cố đã xảy ra"
4. Quan sát	Observation (trong ObservationLedger)	Adapter, qua sổ append-only	Tin ở mức bản ghi gốc đã lưu
5. Kết luận	cạnh VERIFIED, FinalHuntAccount, Disposition	Verifier + controller	Tin trong phạm vi trích dẫn và độ hoàn tất đã ghi

Luật bất biến: **chỉ bộ kiểm tất định (tầng 5) mới nâng một cạnh từ tầng 2 lên "đã chứng minh"**. Mô hình ngôn ngữ dừng ở tầng 2.

4.1.1. Ba trạng thái độc lập của một tuyến ngữ nghĩa

contracts/semantic_route.py tách ba câu hỏi mà một truy vấn rỗng hay bị gộp làm một:

12. **Thực thi xong** — một lần gọi nhà cung cấp có giới hạn đã kết thúc (QueryResult.complete là cờ của lần đó). Số dòng bằng không **không** suy ra hết dữ liệu của mọi tuyến.
13. **Chứng minh xong** — các quan sát được trích dẫn thỏa quan hệ và mọi ràng buộc mà thao tác khai báo là chứng minh được.
14. **Tuyến đã cạn** — mọi giai đoạn truy hồi, mọi trang tiếp, mọi phương án thay đã khai báo đều đã được thử hoặc bị từ chối có lý do kiểm toán.

Đây là nền để chặn lỗi **L3**: một truy vấn rỗng và hoàn tất chỉ chứng tỏ "thực thi xong", chưa chứng minh "sự vắng mặt".

4.1.2. Âm tính hợp lệ

license_valid_negative (m5_adapter/controls.py) chỉ cấp phép một kết quả âm tính khi đồng thời: truy vấn đích chạy được, hoàn tất, không có hàng; **và** ba control đạt — sức khỏe

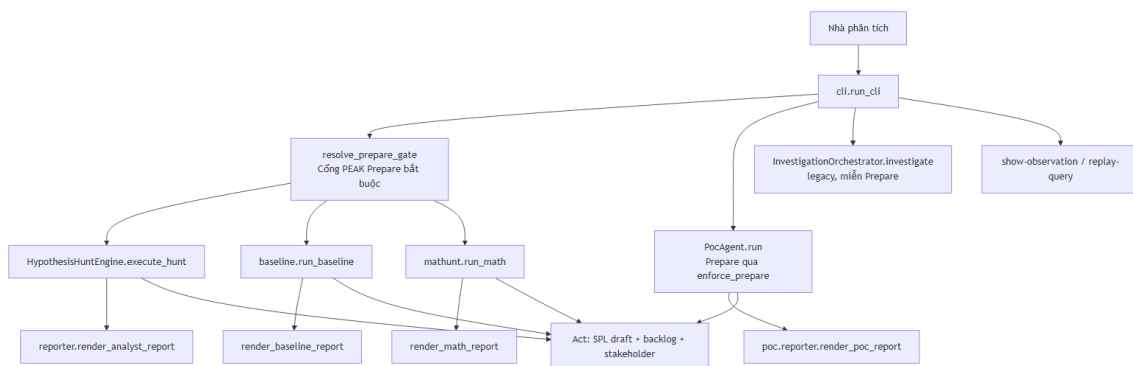
phạm vi, có-bản-ghi-trong-phạm-vi, khả-năng-quan-sát-vị-từ. Thiếu một điều kiện, kết quả ở lại INCONCLUSIVE. HuntOutcome.NO_EVIDENCE_FOUND và Disposition lành tính là hai nhãn khác nhau; báo cáo bị cấm vẽ một truy vấn dở dang thành BENIGN.

4.1.3. Vai trò trường và lời chứng

FieldRole (contracts/case_graph.py) khóa các cặp hay bị lẫn: client_ip ≠ server_ip, endpoint_host ≠ server_host ≠ sensor_host, account_name ≠ person_name, domain_name ≠ IP chưa phân giải. RelationVerifier từ chối dùng một hàng tự khai vai trò máy chủ để chứng minh rằng buộc máy trạm người dùng — tên máy chứa srv không phải bằng chứng vai trò. Lời chứng của người (EpistemicType.TESTIMONY) không bao giờ được nâng thành OBSERVED.

4.2. Ba đường vào, phân nhánh theo tham số

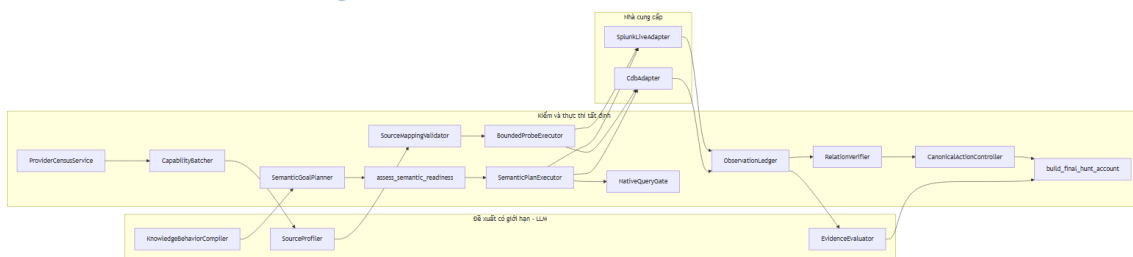
Dòng lệnh hunting.cli.run_cli phân nhánh theo **tham số**, không theo từ khóa trong câu hỏi (chặn lỗi L2).



Hình 4.1. Sơ đồ minh họa

- **Đường sẵn giả thuyết** (HypothesisHuntEngine) là đường kiến trúc v7.
- **Đường cảnh báo** (InvestigationOrchestrator, các gói M1–M5) là vòng đời cũ, đầu vào là Alert.
- **PoC, baseline, math** là chế độ tắt định phục vụ đánh giá và thực hành PEAK.

4.3. Các khối của đường sẵn chuẩn



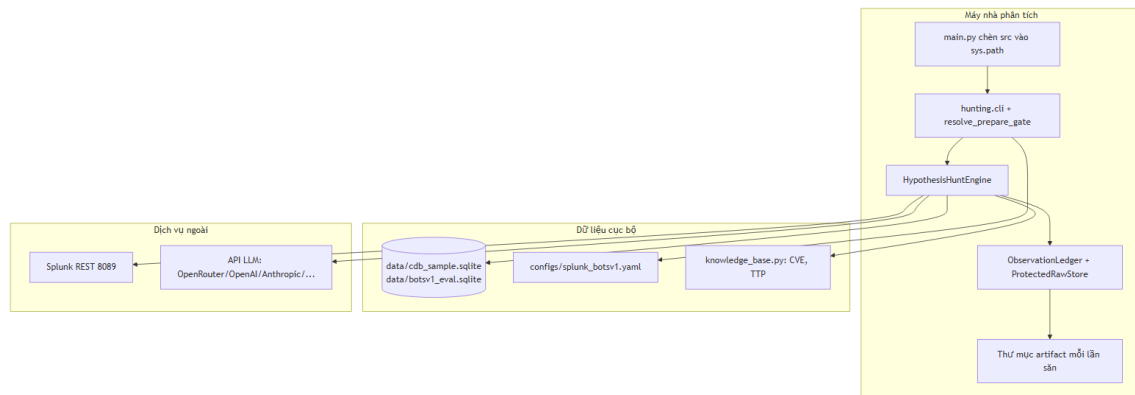
Hình 4.2. Sơ đồ minh họa

Bảng trách nhiệm và ranh giới mô hình:

Bảng 4.2.

Khối	Việc	Ranh giới mô hình
ProviderCensusService	Khám phá phân vùng, schema, quyền, độ hoàn tất	Không gọi mô hình
CapabilityBatcher	Xếp mọi nguồn/trường của một quan hệ vào lô vừa ngữ cảnh	Không gọi mô hình. Điểm số chỉ là thứ tự, không loại nguồn
KnowledgeBehaviorCompiler	CVE/TTP/IOC tất định, hoặc một đề xuất đồ thị cho câu tự do	Một đề xuất đúng schema. Cấm SPL, cấm kết luận
SourceProfiler	Đề xuất nguồn, vai trò trường bằng ID census	Một lần mỗi lô. Cấm bịa ID
BoundedProbeExecutor	Gọi probe của adapter	Tất định. Probe là năng lực quan sát, chưa phải bằng chứng sự cố
SemanticGoalPlanner	Ghép AND phụ thuộc và OR phương án từ hợp đồng kiểu	Tất định. Không đọc tên thao tác như một kịch bản
NativeQueryGate	Nhận/từ chối một SPL ứng viên	Tất định. Ngữ pháp là tập con SPL
ObservationLedger	Ghi quan sát, kết quả truy vấn, ô phủ	Tất định, chỉ thêm (append-only)
RelationVerifier	Cửa nhận trích dẫn; kiểm hợp đồng/chuyển trạng thái/cấp kiểu	Tất định
CanonicalActionController	Một mình đối HuntState và quyết định dừng	Tất định

4.4. Sơ đồ triển khai



Hình 4.3. Sơ đồ minh họa

Secrets của API và mật khẩu Splunk nằm trong môi trường hoặc .env, nạp bởi ApiLLMConfig.from_env; chúng không được ghi vào state hay manifest phát lại.

4.5. Các hợp đồng dữ liệu cốt lõi

Các hợp đồng nằm trong src/hunting/contracts, chủ yếu là dataclass và enum, được kiểm bởi constructor và các hàm parse_and_validate_*.

- **'HuntRequest'** (contracts/hunt.py): id, kind (QUESTION, HYPOTHESIS, TTP, IOC, CVE, CTI_REPORT, SCHEDULED, NL_QUESTION), content, entities, time_policy. Thực thể hạt giống là *chưa kiểm* cho đến khi một quan sát thiết lập quan hệ.

- **`SemanticGoalGraph`** (contracts/semantic_graph.py): SemanticVariable (giá trị có kiểu, không phải trường nhà cung cấp), SemanticConstraint, SemanticRelationGoal, SemanticAnswerGoal, ProofMethod (nhiều method là OR; phụ thuộc của một method là AND). parse_and_validate_semantic_goal_graph từ chối trôi schema.
- **`CapabilityGraph`** (contracts/capabilities.py): ghi nhà cung cấp được chọn, nhà cung cấp bị từ chối và lý do. Một nhà cung cấp tới được nhưng không thỏa claim bị *từ chối*, không dùng làm phương án dự phòng im lặng.
- **`Observation`** (contracts/observations.py): giữ kiểu gốc, trường gốc, Provenance, EpistemicType (OBSERVED/TESTIMONY), nhãn taint, ô, truy vấn. Sự kiện lạ (is_unmapped) vẫn là quan sát hợp lệ, không bị vứt.
- **`StoppingDecision`** và **`HuntOutcome`** (contracts/hunt.py): các quyết định dừng tất định (STOP_RESOLVED, STOP_REFUTED, STOP_INCONCLUSIVE_*, STOP_NEEDS_USER_DECISION, STOP_EXHAUSTED_BY_BUDGET, STOP_UNSUPPORTED_CAPABILITY, STOP_UNREACHABLE, ...) và nhãn nhận thức của giả thuyết (SUPPORTED, CONTRADICTED, INCONCLUSIVE, NO_EVIDENCE_FOUND, ...).

Danh mục đầy đủ 260 lớp và 658 hàm/phương thức, kèm khoảng dòng và docstring, nằm ở Phụ lục A của bản một-tệp (docs/PAPER-BAO-CAO-HE-THONG.md).

Chương 5 — Cổng PEAK Prepare bắt buộc (đóng góp kỹ thuật mới nhất)

Chương này đặc tả đóng góp mới nhất của phiên bản: biến bước Prepare của PEAK từ một ràng buộc chỉ áp cho đường PoC thành một cổng bắt buộc trên mọi đường sẵn. Chương trình bày động cơ, thiết kế, thuật toán resolve, hiện thực trong mã, và kiểm chứng thực nghiệm ba hành vi.

5.1. Động cơ

Trong PEAK, không có cuộc săn nào được bắt đầu mà thiếu bước Prepare — nhà phân tích phải xác định đề tài, mô hình ABLE, phạm vi, thời hạn tối đa và kế hoạch trước khi chạm dữ liệu. Trước phiên bản này, hệ thống chỉ ép Prepare trên **đường PoC** (qua `missing_prepare_fields()` và ngoại lệ `PrepareError` trong `poc/agent.py`). Ba đường sẵn còn lại chạy thẳng, không qua Prepare:

- Đường giả thuyết (`--hypothesis`, `--cve`, `--ttp`, `--ioc`, `--query`, `--threat-actor`, `--campaign`);
- Đường baseline (`--baseline`);
- Đường M-ATH (`--math`).

Điều này mâu thuẫn với nguyên tắc PEAK "không có hunt nào không có Prepare", và làm cho tuyên bố "hệ thống bám PEAK" chỉ đúng một phần.

5.2. Nguyên tắc thiết kế

Yêu cầu đặt ra khi thiết kế cổng:

15. **Bắt buộc trên mọi đường sẵn** (trừ đường cảnh báo legacy — vốn là triage một alert đã có, không có giả thuyết để chuẩn bị).
16. **Không phá luồng chạy hợp lệ.** Một hunt bình thường (ví dụ `--ttp T1190 --time-window ...`) đã mang đủ thông tin để hình thành một Prepare plan tối thiểu; không nên bắt người dùng viết một tệp plan riêng cho mọi lệnh.
17. **Chạy trước khi chạm nhà cung cấp.** Một hunt không hợp lệ về Prepare phải bị chặn *trước* bước environment audit / provider setup, để không phát sinh kết nối telemetry vô ích.
18. **Có van thoát tường minh** cho test/CI và cho các lần chạy ad-hoc có chủ đích.
19. **Minh bạch và audit được.** Plan đã dùng (dù suy tự động) phải được in ra.

5.3. Kiến trúc giải pháp

Ba thành phần được thêm/sửa:

5.3.1. ``peak.py`` — hai hàm mới và một sửa nhỏ

- ``derive_prepare_plan(...)`` — suy một Prepare plan tối thiểu-nhưng-đầy-đủ từ input của hunt. Mọi trường thiếu được điền bằng một giá trị mặc định *tường minh* để plan đầy đủ và nhà phân tích thấy rõ điều gì đã được giả định. `max_duration` được suy từ span của cửa sổ thời gian (nếu có), mặc định 14d. Nếu content (mục tiêu) rỗng, plan cố ý để trống các

trường then chốt để cổng chặn — "một hunt không có mục tiêu không phải là một PEAK hunt".

- ``missing_prepare_fields_from_plan(plan)`` — kiểm checklist PEAK trên một dict plan (đường non-PoC không có object PoC), dùng chung danh sách trường bắt buộc `_REQUIRED` (topic, ABLE behavior/location/evidence, scope, max_duration, plan) cộng `research_refs`. Actor được phép trống — PEAK cho phép actor không rõ.
- ``load_hunt_plan`` được sửa đọc utf-8-sig để chịu được BOM (trình soạn thảo và PowerShell hay thêm BOM), tránh lỗi decode khi nạp `--hunt-plan`.

5.3.2. ``cli.py`` — hàm `resolve` dùng chung và cờ van thoát

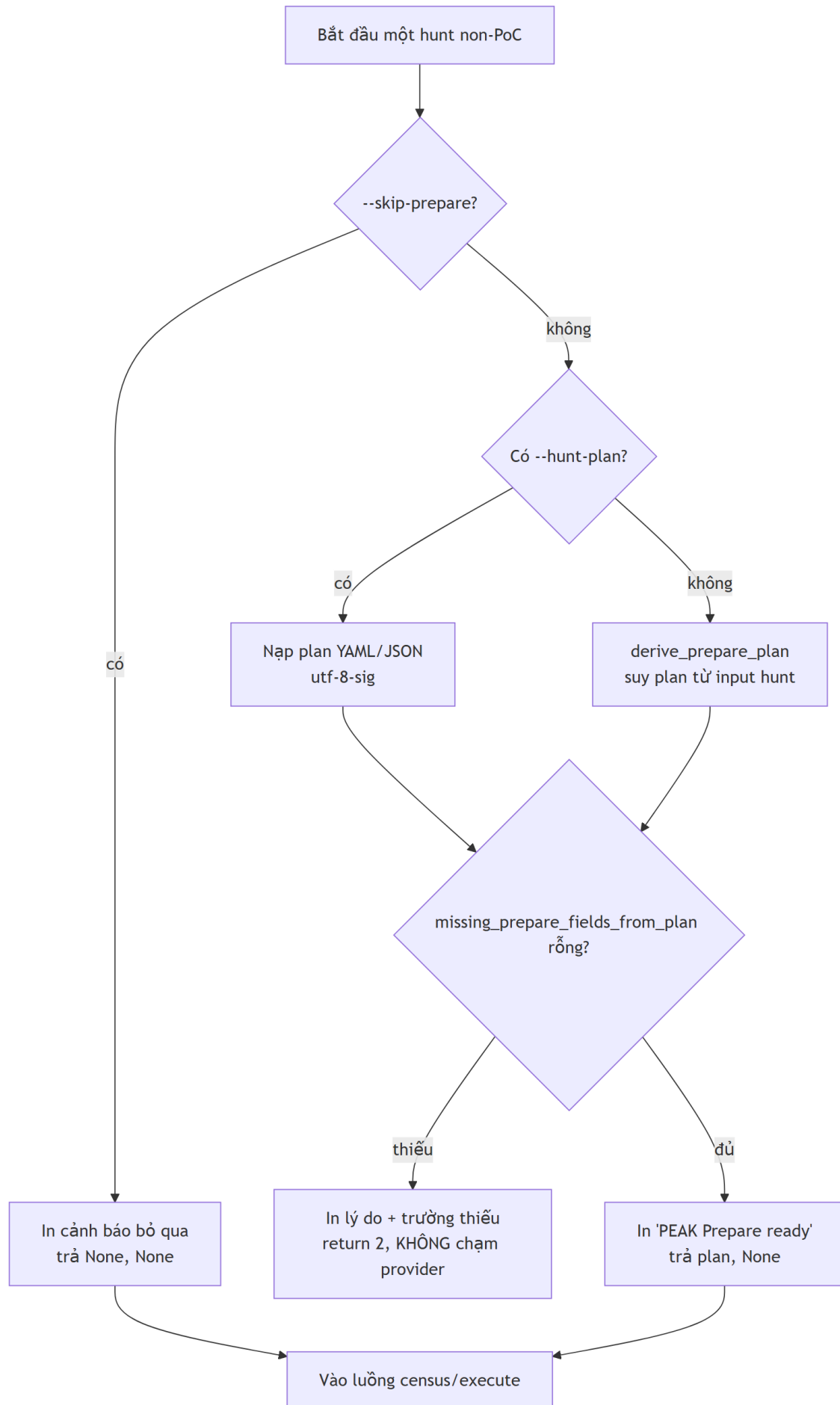
- ``resolve_prepare_gate(args, hunt_kind, content, ...)`` — hàm dùng chung, gọi ở đầu ba dispatch (baseline, math, hypothesis). Trả về (plan, error_code): thành công (plan, None), bị chặn (None, 2), van thoát (None, None).
- ``--skip-prepare`` — cờ mới (mặc định tắt), bỏ qua cổng cho test/CI hoặc lần chạy ad-hoc có chủ đích; khi bật, hunt chạy mà không ghi plan.

5.4. Thuật toán `resolve` của cổng

Thứ tự ưu tiên (điều kiện đầu tiên khớp thì thắng):

20. `--skip-prepare` bật → in cảnh báo bỏ qua, trả (None, None), hunt chạy tiếp không ghi plan.
21. Có `--hunt-plan` → nạp plan tường minh. Một plan tường minh **thiếu trường** là lỗi cứng → (None, 2): nhà phân tích đã yêu cầu một plan, và nó phải là một PEAK plan hợp lệ.
22. Không có plan → **suy** một plan tối thiểu từ input hunt. Nếu plan suy ra vẫn thiếu (ví dụ không có mục tiêu) → (None, 2).

Thành công thì in `[+] [PREPARE] PEAK Prepare ready: ...` rồi trả (plan, None).



Hình 5.1. Sơ đồ minh họa

Điểm then chốt: cổng được đặt **trước** environment audit trong run_cli, nên một hunt bị chặn không hề gửi yêu cầu tới Splunk hay mở SQLite.

5.5. Trích đoạn hiện thực (rút gọn)

Lỗi của hàm suy plan (peak.py), cho thấy cách max_duration suy từ cửa sổ và cách trường thiếu được điền tường minh:

```
def derive_prepare_plan(*, hunt_kind, content, scope="", time_window="",
                        evidence_hint="", location_hint="", ...):

    kind = str(hunt_kind or "hunt").strip() or "hunt"

    goal = str(content or "").strip()

    topic = goal if goal else ""

    behavior = goal if goal else ""

    location = str(location_hint or "").strip() or (
        f"telemetry in scope {scope}" if scope.strip() else "in-scope telemetry")

    ...

    # PEAK max duration vừa là thời hạn hunt. Suy từ span của sổ nếu có.

    if win and "/" in win:

        start, end = validate_time_window_format(win)

        days = max(1, (end - start).days or 1)

        max_duration = f"{days}d"

    ...

    # content rỗng -> topic/behavior/evidence/plan để trống -> cổng chặn
```

Hàm resolve (cli.py), cho thấy ba nhánh và điểm chặn trả 2:

```
def resolve_prepare_gate(args, *, hunt_kind, content, ...):

    if getattr(args, "skip_prepare", False):

        print("[!] [PREPARE] PEAK Prepare gate skipped (--skip-prepare); no plan
        recorded.")

        return None, None

    if plan_path:

        # --hunt-plan tường minh

        plan = load_hunt_plan(plan_path); explicit = True
```

```

else:
    # suy tự động

    plan = derive_prepare_plan(hunt_kind=hunt_kind, content=content, ...)

    missing = missing_prepare_fields_from_plan(plan)

    if missing:

        print(f"[-] [PREPARE] PEAK Prepare is incomplete ... (missing: {'',
        '.join(missing)}) ...")

        return None, 2

    print(f"[+] [PREPARE] PEAK Prepare ready: topic={plan.get('topic')!r} ...")

    return plan, None

```

Cổng được nối vào ba dispatch bằng ba lời gọi tương tự, ví dụ ở baseline:

```

_plan, _gate = resolve_prepare_gate(args, hunt_kind="baseline",

    content=f"baseline survey (EDA) of {data_source}", scope=..., time_window=window,
    ...)

if _gate is not None:

    return _gate

```

Ở đường giả thuyết, cổng đặt ngay sau khi xác định `is_hypothesis_hunt` và trước `environment` audit; đường cảnh báo legacy được miễn.

5.6. Kiểm chứng ba hành vi (đo trong phiên báo cáo)

Bảng 5.1.

#	Kịch bản	Lệnh	Kết quả đo
1	Hunt bình thường (plan suy tự động)	--hypothesis "Amber Turing visited a competitor website..." -- provider cdb	In [+] [PREPARE] PEAK Prepare ready: topic='Amber Turing visited...' scope='hypothesis over the selected telemetry' max_duration='14d' (source=derived from hunt inputs), hunt chạy tiếp bình thường (INCONCLUSIVE)
2	--hunt-plan thiếu trường	--baseline cdb:events --hunt- plan bad.json (plan chỉ có topic)	Exit 2 , báo missing: able.behavior, able.location, able.evidence, scope, max_duration, plan,

			research_refs
3	Van thoát	--baseline cdb:events --skip-prepare	Exit 0 , in [!] [PREPARE] PEAK Prepare gate skipped, baseline chạy bình thường

Ngoài ra, kiểm chứng phát hiện một lỗi phụ đã được sửa cùng đóng góp này: load_hunt_plan trước đó đọc utf-8, gặp BOM do PowerShell/editor thêm vào thì báo Unexpected UTF-8 BOM; sửa thành utf-8-sig khắc phục.

5.7. Không phá luồng cũ

Sau thay đổi, toàn bộ **514 test đạt / 13 bỏ qua / 0 thất bại** (chạy lại hai lần trong phiên báo cáo). Thiết kế đảm bảo điều này bằng ba cơ chế:

23. Các test unit của baseline/math gọi thẳng run_baseline/run_math (không qua CLI), nên không chạm cổng.
24. Các test CLI đường giả thuyết (--cve, --ttp, --query) không truyền --hunt-plan, nhưng cổng **suy tự động** một plan đầy đủ từ chính các cờ đó, nên vẫn chạy tiếp (exit 0) như trước.
25. --skip-prepare là van thoát cho bất kỳ test nào cần bỏ qua tường minh.

5.8. Vị trí trong tài liệu và mã

- Đặc tả chi tiết: docs/PEAK-PREPARE-GATE.md.
- Mã: src/hunting/peak.py (derive_prepare_plan, missing_prepare_fields_from_plan, load_hunt_plan sửa BOM), src/hunting/cli.py (resolve_prepare_gate, cờ --skip-prepare, ba lời gọi ở baseline/math/hypothesis).

Chương 6 — Phương pháp thực nghiệm

Chương này mô tả môi trường thực nghiệm, bộ dữ liệu Boss of the SOC v1 (BOTS v1), thiết kế bốn lớp kiểm thử, và tiêu chí đo. Kết quả cụ thể nằm ở Chương 7.

6.1. Mục tiêu thực nghiệm

Thực nghiệm nhằm trả lời câu hỏi nghiên cứu CH3 (Chương 1): trên dữ liệu tấn công thật, hệ thống có đạt độ đặc hiệu cao (ít dương tính giả) mà vẫn bắt được tấn công thật (có độ nhạy) hay không, và đo được đến đâu. Đồng thời, thực nghiệm kiểm chứng rằng các ràng buộc kiến trúc (Chương 4) và cổng Prepare mới (Chương 5) không làm hồi quy hành vi phần mềm.

Bốn đại lượng đo:

- **D1 — Tính đúng đắn phần mềm:** bộ kiểm thử tự động đạt/thất bại.
- **D2 — Độ đặc hiệu (specificity):** tỉ lệ dương tính giả khi chạy PoC trên telemetry lành tính quy mô lớn.
- **D3 — Độ nhạy (recall) trên tấn công thật:** hệ thống có bắt được một tấn công đã biết hay không.
- **D4 — Hành vi cổng Prepare:** ba kịch bản (suy tự động / plan thiếu / van thoát).

6.2. Môi trường thực nghiệm

Bảng 6.1.

Thành phần	Cấu hình
Hệ điều hành	Windows (win32), PowerShell
Python	3.12 (yêu cầu tối thiểu 3.10)
Thư viện runtime	Pydantic 2, PyYAML, requests
Công cụ phát triển	pytest, pytest-cov, ruff
Mô hình ngôn ngữ	meta/muse-spark-1.3-contributor qua OpenRouter (cấu hình trong .env)
Backend telemetry	SQLite (CDB) và Splunk REST (khi có)

Cài đặt và kiểm nhanh (theo README.md):

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
python -m pytest tests/unit -q
python -m compileall -q src main.py
ruff check .
```

6.3. Vì sao chọn BOTS v1

Boss of the SOC v1 (BOTS v1) là bộ dữ liệu telemetry công khai của Splunk, mô phỏng một chuỗi tấn công thật trên trang imreallynotbatman.com: quét và khai thác Joomla (T1190), brute-force, lateral movement, và C2. Lý do chọn:

- 26. **Có ground-truth công khai** qua walkthrough của Splunk, nên xác minh được kết quả.
- 27. **Vừa có tấn công vừa có nhiều hành tính**, cho phép đo cả độ nhạy (trên tấn công) lẫn độ đặc hiệu (trên nhiễu) trên cùng một bộ.
- 28. **Là chuẩn thực địa** được cộng đồng sẵn mỗi đe dọa dùng rộng rãi.

6.4. Hai backend thực thi

Bảng 6.2.

Backend	Tệp / kết nối	Vai trò trong thực nghiệm
CDB (SQLite)	data/cdb_sample.sqlite, data/botsv1_eval.sqlite	Backend chính cho đánh giá tự động — phát lại được, chạy không cần Splunk
Splunk sống	SplunkLiveAdapter qua REST cổng 8089	Môi trường SIEM thật, khi có

Ranh giới trung thực: một kết quả trên CDB chứng minh hợp đồng và logic bằng chứng, **không** chứng minh hành vi của một SIEM sản xuất. Đây là lý do các test cần Splunk sống bị bỏ qua (skip) khi không có instance.

6.5. Hai bộ dữ liệu

6.5.1. Bộ sample — 16 sự kiện

data/cdb_sample.sqlite, gieo bằng `scripts/seed_botsv1_sample.py`, phủ các chặng chính của BOTS v1 để demo end-to-end nhanh:

Bảng 6.3.

Thời điểm (2016-08-21)	Sự kiện	MITRE
03:00–03:06	7 logon thất bại vào we1149srv (brute-force)	T1110
03:08	alice logon thành công (pivot)	T1078
06:02	email phishing SMTP	T1566.001
08:14	beacon tới ad.networkfilter.co	T1071.001
08:14	powershell -enc con của PDF exploit	T1059.001 + T1027
11:33	lateral SMB tới JGREEN-PC	T1021.002
11:42	scheduled task + Run-key persistence	T1053.005 / T1547.001
02:15	script SCCM admin hành tính (ứng viên FP)	—

6.5.2. Bộ eval — 4.42 triệu dòng

data/botsv1_eval.sqlite (gitignored, ~6 phút để nạp), tạo bằng scripts/ingest_botsv1_eval.py và scripts/ingest_http.py:

Bảng 6.4.

Nguồn	Nội dung	Số dòng
WinEventLog:Security.csv.gz (lọc giữ auth/process/SMB)	4624 logon, 4688 process, 5140/5145 SMB, 4648 explicit logon	4,342,563
Sysmon Event 1 (process create)	tiến trình thật (WmiPrivSE, wormgr, rundll32...)	66
stream:dns	query thật (PTR, NBNS, wpad, crl.microsoft.com...)	35,904
stream:http	web attack thật: Joomla RCE trên imrealllynotbatman.com (~19.7k) + benign (windowsupdate/msn/google)	39,010
Tổng eval DB		4,417,543

Timeline: 2016-08-10 → 2016-08-28 (19 ngày, đúng window BOTS v1). Cửa sổ dùng cho đánh giá độ đặc hiệu là 2016-08-10 → 2016-08-28, tổng 4,378,533 dòng nằm trong cửa sổ đó.

6.6. Thiết kế bốn lớp kiểm thử

Hệ thống có bốn lớp kiểm thử, tương ứng bốn mức tin cậy khác nhau.

6.6.1. Lớp 1 — Unit test hợp đồng (74 tệp → D1)

Khóa từng hợp đồng và từng pha. Nhóm tiêu biểu:

Bảng 6.5.

Tệp	Khóa điều gì
test_phase0_contracts.py, test_contracts.py	Hợp đồng nền
test_phase1_compiler.py, test_v6_semantic_claim_compiler.py	Biên dịch và từ chối SPL (chặn L1)
test_provider_census.py, test_capability_retriever.py	Census và batch không Top-K (chặn L2)
test_semantic_goal_planner.py, test_semantic_executor.py, test_semantic_readiness.py	Plan, thực thi, sẵn sàng chứng minh
test_v5_relation_verifier.py, test_deterministic_transition_verifier.py	Cạnh và chuyển trạng thái
test_counterfactual_matrix.py, test_route_negative_license_and_provenance.py	Cùng từ khóa cho đồ thị khác nhau; âm tính không cấp phép bừa (chặn L3)
test_security_regression.py, test_llm_timeout_fail_fast.py	Injection và hết giờ

Lệnh: python -m pytest tests/unit -q.

6.6.2. Lớp 2 — Integration (1 tệp)

tests/integration/test_botsv1_web_compromise.py chạy lát cắt đọc BOTS v1 end-to-end.

6.6.3. Lớp 3 — Đánh giá độ đặc hiệu (→ D2)

Chạy bốn PoC dựng sẵn trên toàn eval DB, đo dương tính giả:

```
.venv\Scripts\python.exe scripts/run_fp_eval.py # ~30s, ghi data/eval_fp_results.json
```

Định nghĩa đo: telemetry lành tính = toàn bộ eval DB (trong đó gần như không có các chằng mà PoC dựng sẵn nhắm tới); dương tính giả = một PoC dựng sẵn báo MATCHED trên dữ liệu này. FP rate = số dòng match / tổng dòng.

6.6.4. Lớp 4 — Đánh giá độ nhạy trên tấn công thật + PoC end-to-end (→ D3)

Chạy PoC Joomla RCE trên stream:http thật, kèm LLM judge:

```
.venv\Scripts\python.exe main.py --provider cdb --db data/botsv1_eval.sqlite `
--poc-file pocs/poc-joomla-rce.json `
--time-window "2016-08-10T21:36:00Z/2016-08-10T22:00:00Z" `
--poc-judge --llm api --poc-judge-max-tokens 16000
```

Để kiểm rằng judge không bịa dương tính khi không có tấn công, chạy thêm ba PoC âm tính (poc-pdf-exploit-enc, poc-bruteforce-we1149srv, poc-c2-beacon-networkfilter) trên cùng eval DB.

6.7. Tiêu chí đo và tính lặp lại

- **D1** đạt khi 0 test thất bại (skip do thiếu Splunk/mạng không tính là thất bại).
- **D2** đo bằng FP rate; mục tiêu 0.
- **D3** đo bằng verdict rules (MATCHED/EMPTY) là chính, judge chỉ advisory. Judge LLM non-deterministic nên verdict chính luôn là rules.
- **D4** đo bằng ba kịch bản cổng Prepare (Chương 5).

Mọi lần chạy PoC sinh một ledger JSON và một report Markdown tái chạy được; các lệnh tái chạy đầy đủ nằm trong docs/EVAL-GROUND-TRUTH.md và docs/BOTS-V1-WALKTHROUGH.md.

Chương 7 — Kết quả và phân tích tỉ lệ

Mọi con số trong chương này truy được về một tệp nguồn trong kho hoặc một lần chạy đã ghi lại trong phiên báo cáo. Phần chưa đo được nói rõ là chưa đo. Bốn đại lượng D1–D4 tương ứng bốn lớp thực nghiệm ở Chương 6.

7.1. D1 — Tính đúng đắn phần mềm: 514 passed, 13 skipped, 0 failed

Chạy `pytest tests/unit -q` hai lần trong phiên báo cáo (mỗi lần ~65 giây):

514 passed, 13 skipped, 21 warnings in 65.xx s

- **0 test thất bại.**
- **13 test bỏ qua** là các test cần Splunk sống hoặc mạng ngoài (ví dụ live Splunk adapter, phase D live) — không phải thất bại.
- `compileall -q src main.py` và `ruff check` trên mã nguồn lỗi (`src/hunting/peak.py`, `src/hunting/cli.py`): sạch, 0 lỗi.

Kết quả này được đo **sau** khi tích hợp cổng PEAK Prepare mới (Chương 5), xác nhận cổng không gây hồi quy.

So với mốc lịch sử: checklist ghi 356 passed / 3 failed / 1 skipped vào 2026-09-09. Con số 514 hiện tại thay cho mốc đó; ba thất bại cũ (test từ khóa và đồ thị email đời v5) không còn.

7.2. D2 — Độ đặc hiệu: 0 dương tính giả trên 4.38 triệu dòng

Nguồn: `data/eval_fp_results.json`. Cửa sổ 2016-08-10 → 2016-08-28, tổng 4,378,533 dòng, tổng thời gian 26.6 giây.

Bảng 7.1.

PoC	Verdict	Quan sát	FP rate	Thời gian (s)
poc-phishing-powershell-enc	EMPTY	0	0.000000	10.77
poc-c2-beacon	EMPTY	0	0.000000	4.73
poc-office-macro	EMPTY	0	0.000000	4.71
poc-credential-phish	EMPTY	0	0.000000	5.76

Độ đặc hiệu = 100% trên 4.38 triệu dòng lành tính (0 dương tính giả).

7.2.1. Hai lỗi được khai ra và sửa

Đánh giá này khai ra hai lỗi (nay đã sửa), minh họa giá trị của việc đo trên quy mô lớn:

29. **`EQUALS` qua `LIKE` (nghiêm trọng):** image EQUALS powershell.exe match cả splunk-powershell.exe → 100 dương tính giả trên 4.38 triệu dòng. Sửa: post-filter khớp chính xác + fallback theo basename (C:\...\powershell.exe vẫn khớp, splunk-powershell.exe không khớp), áp cho mọi toán tử trong agent._run_step.
30. **Step `EXISTS` với value rỗng (trung bình):** image EXISTS value="" match dòng đầu tiên có trường bất kỳ. Sửa: EXISTS rỗng → 0 dòng; đồng thời xóa một step vô nghĩa khỏi poc-c2-beacon.

Trước khi sửa: FP rate 0.000023 (100 dòng). **Sau khi sửa: 0.** Ba test hồi quy được thêm để khóa hành vi này.

7.3. D3 — Độ nhạy trên tấn công thật: Joomla RCE MATCHED

Tấn công thật trong stream:http (39k dòng, Joomla ~19.7k): imreallynotbatman.com ~22k hit, /joomla/index.php/component/search/ ~16.6k hit — đúng chặng web-compromise của BOTS v1 (quét + khai thác Joomla).

PoC poc-joomla-rce.json (domain EQUALS imreallynotbatman.com + uri CONTAINS /joomla/), chạy trong phiên báo cáo:

PoC poc-joomla-rce – verdict MATCHED – 199 obs, 2 matched step(s),

1 LLM call(s) [match=0, judge=1], 5163 tokens | JUDGE: TRUE_POSITIVE (0.85)

- **Rules bắt được attack thật (recall bước match = có):** 19.7k dòng Joomla → 199 quan sát (giới hạn 100/bước). Benign cùng tệp (windowsupdate/msn) không match.
- **Judge phân biệt đúng recon vs compromise:** nhận ra Acunetix scanner probe (/acunetix-wvs-test-for-some-inexistent-file), các path fuzz ngẫu nhiên (/PdgyH6M, /OD6xDhbF), và Joomla enumeration → kết luận đây là giai đoạn quét chủ động, không phải duyệt web bình thường.
- **Đây là dương tính thật đầu tiên trên tấn công thật** (không phải sample tự dựng).

7.3.1. Tính non-deterministic của judge

Cùng một bằng chứng Joomla cho các nhãn/độ tự tin khác nhau qua các lần chạy:

Bảng 7.2.

Nguồn lần chạy	Judge verdict	Độ tự tin
EVAL-GROUND-TRUTH.md (bản ghi cũ)	INCONCLUSIVE	0.82
Phiên báo cáo (lần 1)	TRUE_POSITIVE	0.87
Phiên báo cáo (lần 2)	TRUE_POSITIVE	0.85

Đây chính là lý do **verdict chính luôn là rules tất định**, còn judge chỉ advisory (dùng để phân biệt recon vs compromise — việc rules không làm được).

7.4. Judge trên các case âm tính: NO_SIGNAL đúng

Ba PoC âm tính chạy trên cùng eval DB trong phiên báo cáo:

Bảng 7.3.

PoC	Verdict (rules)	Quan sát	LLM calls	Judge
poc-pdf-exploit-enc	EMPTY	0	0	NO_SIGNAL
poc-bruteforce-we1149srv	EMPTY	0	0	NO_SIGNAL
poc-c2-beacon-networkfilter	EMPTY	0	0	NO_SIGNAL

Judge giữ im lặng đúng ("absence of evidence, not evidence of absence"), **không bịa dương tính**, và **không tốn token** — khi không có hàng, judge từ chối gọi LLM. Đây là bằng chứng trực tiếp rằng hệ thống chặn lỗi L3 ở tầng hành vi thực tế, không chỉ ở đơn vị.

7.5. D4 — Hành vi cổng PEAK Prepare

Bảng 7.4.

Kịch bản	Kết quả đo
Hunt bình thường (plan suy tự động)	In PEAK Prepare ready ... (source=derived from hunt inputs), hunt chạy tiếp
--hunt-plan thiếu trường	Exit 2, báo missing: able.behavior, able.location, ... research_refs
--skip-prepare	Exit 0, in PEAK Prepare gate skipped, hunt chạy

Xem Chương 5 để có chi tiết đầy đủ.

7.6. Demo end-to-end trên sample 16 sự kiện

Chuỗi ba bước trên data/cdb_sample.sqlite:

Bảng 7.5.

Bước	Lệnh	Kết quả đo
Baseline (EDA, không LLM)	--baseline cdb:events	16 dòng → 11 trường, 23 outlier, 4 gap (~0.0002s)
M-ATH (--math)	--math cdb:events	25 lead; encoded PowerShell (score 6.50) xếp trên brute-force rồi rác (5.81)
PoC + Judge	--poc-file poc-pdf-exploit-enc.json --poc-judge --llm api	MATCHED 9 obs, 3 bước; judge INCONCLUSIVE 0.88 (bóc được SCCM foobar là noise; 2 cmdline alice thiếu parent/network nên không kết TP)

Baseline phát hiện outlier hợp lý (smtp, schtasks.exe, svc_admin, SCCM-SERVER) và gap đúng (domain, file_path, action, status trống trong window). M-ATH xếp đúng thứ tự ưu tiên (encoded PowerShell lên trước). Judge INCONCLUSIVE 0.88 là đúng chuẩn khi thiếu bằng chứng parent/network.

7.7. Bảng tổng hợp tỉ lệ

Bảng 7.6.

Đại lượng	Giá trị đo	Nguồn
D1 — Test phần mềm đạt	514/527 (13 skip), 0 fail	phiên báo cáo
D2 — Độ đặc hiệu (FP trên benign)	100% (0 FP / 4,378,533 dòng)	eval_fp_results.json
D3 — Recall bước match (Joomla thật)	có (199 obs, 2/2 bước)	EVAL-GROUND-TRUTH.md + phiên báo cáo
Judge trên 4 case (phiên báo cáo)	1 TP đúng, 3 NO_SIGNAL đúng	phiên báo cáo
D4 — Cổng Prepare	3/3 kịch bản đúng	phiên báo cáo
F1 tổng quát (claim/cạnh/câu trả lời)	chưa đo — thiếu nhãn đầy đủ	04-IMPLEMENTATION-CHECKLIST.md

Diễn giải và hạn chế của các con số này nằm ở Chương 8.

Chương 8 — Bàn luận và hạn chế

Chương này diễn giải ý nghĩa của kết quả (Chương 7) trong khung câu hỏi nghiên cứu (Chương 1), rồi nêu các hạn chế trung thực của kho tại thời điểm báo cáo.

8.1. Diễn giải kết quả theo câu hỏi nghiên cứu

8.1.1. CH1 — Logic tất định giữ quyền quyết định

Kết quả D2 và mục 7.4 ủng hộ CH1. Trên 4.38 triệu dòng, verdict luôn do rules quyết định (MATCHED/EMPTY), và judge LLM chỉ chạy sau khi đã có hàng; khi không có hàng, judge từ chối gọi LLM và trả NO_SIGNAL với 0 token. Tính non-deterministic của judge (mục 7.3.1) — cùng bằng chứng cho 0.82/0.85/0.87 — cho thấy rõ vì sao judge **không được** làm verdict chính: nếu để LLM quyết định, kết luận sẽ dao động giữa các lần chạy. Kiến trúc đặt LLM ở tầng đề xuất (tầng 2, Chương 4) là câu trả lời trực tiếp cho CH1.

8.1.2. CH2 — Ảnh xạ PEAK mà không đánh mất bản chất

Chương 3 và Chương 5 ủng hộ CH2. Hệ thống hiện thực hóa bốn cửa PEAK trên đường PoC, và với đóng góp mới, ép Prepare thành cổng bắt buộc trên mọi đường sẵn. Điểm tinh tế: cổng **suỵ plan tự động** khi hunt đã mang đủ mục tiêu, thay vì bắt người dùng viết plan cho mọi lệnh — điều này giữ được tinh thần PEAK "không hunt nào không có Prepare" mà không biến quy trình thành gánh nặng thủ tục. Kết quả D4 (3/3 kịch bản đúng) và D1 (0 hồi quy) xác nhận cổng hoạt động đúng và không phá luồng cũ.

8.1.3. CH3 — Độ đặc hiệu cao, độ nhạy có, đo được một phần

Kết quả D2 (0 FP) và D3 (Joomla MATCHED) cho thấy hệ thống phát hiện được tấn công thật mà không báo bừa trên nhiều lành tính — trả lời khẳng định cho CH3 *trong phạm vi đã đo*. Tuy nhiên, "đo được đến đâu" là phần cần trung thực: recall mới đo được **một chặng** (Joomla web), và không có số F1 tổng quát. Chi tiết ở mục 8.2.

8.2. Hạn chế trung thực

Các hạn chế sau là sự thật của kho tại thời điểm báo cáo, bám sát checklist và mã. Chúng được nêu để báo cáo không quá đà.

31. **Recall mới đo được một chặng.** Chỉ chặng Joomla web đo được recall. Các chặng khác (failed-logon 4625, encoded PowerShell, beacon networkfilter) hiện **không có trong dữ liệu đã tải** — chúng nằm ở sourcetype (Sysmon full) chưa nạp. Việc quét chuỗi con từng báo sai đã được đính chính: 4656 bị nhầm là 4625 (do RecordNumber chứa "4625"), iexplore.exe bị nhầm là IEX. Tức là **0x failed-logon 4625 thật, 0x encoded-PS thật, 0x beacon `networkfilter` thật** trong dữ liệu hiện có.
32. **Judge LLM non-deterministic.** Cùng bằng chứng cho các độ tự tin khác nhau qua các lần chạy (đã ghi 0.82 / 0.85 / 0.87 / 0.88, và các bản cũ tới 0.92 / 0.97). Judge chỉ là advisory; verdict chính luôn là rules tất định.

33. **F1 chưa đo.** Precision/recall/F1 của claim, cạnh và câu trả lời chưa được đo trên bộ có nhãn đầy đủ. Không có con số F1 tổng quát nào trong báo cáo này. Chi phí USD là ước lượng từ bảng giá cục bộ và số token, không phải hóa đơn.
34. **Hai backend, một dataset.** Chỉ SQLite (CDB) và Splunk. EDR/IDS/thư chỉ thành nguồn khi có adapter và kiểm thử năng lực tương ứng. Khái quát từ BOTS v1 sang mọi SIEM chưa được tài liệu kiến trúc cho phép.
35. **Baseline window ngắn.** Demo baseline chạy trên một ngày; PEAK khuyến nghị cửa sổ 30–90 ngày cho hầu hết nguồn.
36. **`VALIDATED` chỉ là parser chấp nhận cú pháp.** SPL đạt trạng thái VALIDATED nghĩa là parser Splunk chấp nhận cú pháp đọc-only, không phải một detection đã phát hành với tỉ lệ dương tính giả đã đo.
37. **Cổng SPL chỉ nhận một tập con SPL.** NativeQueryGate cố ý không parse toàn bộ SPL; an toàn với tập con đó không phải an toàn với mọi SPL Splunk.
38. **Hai thể hệ mã cùng tồn tại.** Đường v7 (SemanticGoalGraph) và đường tương thích cũ (discovery/adaptive, ClaimGraph) cùng nằm trong execute_hunt. Checklist còn mục gỡ ngôn ngữ template và các nhánh từ khóa còn sót; đọc một hàm đòi cũ mà tưởng nó là cửa duy nhất của mọi câu hỏi sẽ hiểu sai hệ thống.

8.3. Đe dọa tới tính hợp lệ (threats to validity)

- **Tính hợp lệ nội tại:** đánh giá độ đặc hiệu dùng định nghĩa "benign = toàn bộ eval DB", trong đó gần như không có các chằng mà PoC dựng sẵn nhắm tới. Do đó 0 FP là kết quả đúng nhưng cần đọc kèm ngữ cảnh: nó đo tính "không báo bừa trên nhiều", không phải "không bỏ sót tấn công".
- **Tính hợp lệ ngoại tại:** kết quả trên CDB (SQLite) chưa khái quát sang Splunk sản xuất; một lần chạy lại chỉ chứng minh đúng hành vi đã đo của lần đó.
- **Tính hợp lệ cấu trúc:** judge non-deterministic khiến bất kỳ chỉ số nào dựa trên judge đều phải đọc như advisory, không như số đo cứng.

8.4. So sánh với kỳ vọng ban đầu

Ba lỗi cố hữu (L1–L3, Chương 1) được chặn ở cả tầng thiết kế lẫn tầng hành vi đo được:

Bảng 8.1.

Lỗi	Cơ chế chặn (thiết kế)	Bằng chứng hành vi (đo)
L1 — mô hình viết thẳng truy vấn	Lớp ngữ nghĩa có kiểu + NativeQueryGate	Test biên dịch từ chối SPL; verdict do rules
L2 — thiên kiến kịch bản theo từ khóa	Phân nhánh theo tham số; batch không Top-K	test_counterfactual_matrix.py; nguồn chọn theo hợp đồng kiểu
L3 — đọc rỗng thành lành tính	Ba trạng thái tuyến + âm tính phải cấp phép	3 PoC âm tính → NO_SIGNAL, 0 token (mục 7.4)

Chương 9 — Kết luận và hướng phát triển

9.1. Kết luận

Báo cáo này trình bày AI Agent Hunting, một tác tử điều tra mối đe dọa bị ràng buộc bởi bằng chứng và vận hành theo khung PEAK của Splunk. Đóng góp trung tâm về mặt kiến trúc là một ranh giới nhận thức chặt: **mô hình ngôn ngữ chỉ đề xuất; census, probe, adapter, sổ cái append-only, bộ kiểm chứng quan hệ và bộ điều khiển tắt định mới giữ quyền với sự thật, truy vấn và quyết định dừng**. Đơn vị lập luận là một đồ thị mục tiêu có kiểu, gắn vào năng lực nhà cung cấp đã kiểm, rồi đối chiếu với quan sát có trích dẫn.

Hệ thống đáng tin ở chỗ nó từ chối hai phép biến đổi nguy hiểm: biến một câu tiếng người thành một câu chuyện tấn công (chặn thiên kiến kịch bản), và biến một truy vấn rỗng thành một kết luận âm (phân biệt "không có bằng chứng" với "có bằng chứng về sự vắng mặt").

Đóng góp mới nhất của phiên bản — **cổng PEAK Prepare bắt buộc trên mọi đường sẵn** — khép lại một khoảng cách giữa tuyên bố "bám PEAK" và hiện thực: nay không đường sẵn nào (giả thuyết, baseline, M-ATH) chạy mà thiếu một Prepare plan, dù là plan tường minh hay plan suy tự động được in ra để audit.

Về mặt thực nghiệm, trên bộ dữ liệu công khai BOTS v1, hệ thống đạt:

- **514 test đạt / 13 bỏ qua / 0 thất bại** — tính đúng dẫn phần mềm sau khi tích hợp cổng mới.
- **0 dương tính giả trên 4.38 triệu dòng lành tính** — độ đặc hiệu 100% trong phạm vi đo.
- **Một dương tính thật đầu tiên** trên tấn công Joomla RCE (MATCHED, 199 quan sát, judge TRUE_POSITIVE).

Những kết quả này cho thấy hệ thống phát hiện được tấn công thật mà không báo bừa trên nhiều lành tính, trong phạm vi dữ liệu đã đo. Đồng thời, báo cáo giữ trung thực về những gì chưa đo được: recall mới đo một chặng, F1 tổng quát chưa có do thiếu nhãn.

9.2. Hướng phát triển

Theo thứ tự ưu tiên, bám các mục còn mở trong 04-IMPLEMENTATION-CHECKLIST.md:

39. **Thêm sourcetype để đo các chặng còn lại.** Nạp Sysmon full và các nguồn còn thiếu để đo recall trên failed-logon (4625), encoded PowerShell, và beacon C2 — hiện các chặng này không có trong dữ liệu đã tải.
40. **Đo F1 trên dữ liệu có nhãn.** Xây bộ nhãn cho claim, cạnh và câu trả lời để đo precision/recall/F1 thay vì chỉ đo độ đặc hiệu và recall bước match.
41. **Hoàn tất cổng sản xuất Splunk.** Đưa SplunkLiveAdapter qua các cổng bằng chứng đầy đủ để một kết quả trên Splunk sống có cùng mức tin như trên CDB.
42. **Khép các đường tương thích cũ.** Gỡ ngôn ngữ template và các nhánh từ khóa còn sót, để đường v7 (SemanticGoalGraph) là cửa duy nhất cho mọi câu hỏi.
43. **Mở rộng nhà cung cấp.** Thêm adapter EDR/IDS/thư với census, probe, control và bộ kiểm năng lực — executor và verifier không nhận thêm nhánh theo tên hãng.

44. **Siết PEAK Prepare sâu hơn (tùy chọn).** Ví dụ bắt buộc `research_refs` do người dùng cung cấp thay vì cho suy mặc định, hoặc ghi plan đã-suy vào artifact của hunt để lưu vết đầy đủ.

9.3. Lời kết

Cho đến khi các mục trên hoàn tất, một lần sẵn thành công trong hệ thống này là **một tài khoản có trích dẫn hoặc một kết quả không kết luận có lý do** — không phải một đoạn văn trông hợp lý. Đó là tiêu chí thành công mà một tác tử điều tra an ninh nên hướng tới: đúng dẫn về bằng chứng quan trọng hơn trôi chảy về ngôn từ.

Chương 10 — Tài liệu tham khảo

*Phân nhóm theo chủ đề. Mã REF- * khớp bảng truy vết trong 03_LITERATURE-AND-TRACEABILITY.md. Ghi chú mức độ tin cậy ở cuối chương.*

10.1. Khung PEAK và thực hành sẵn mỗi đe dọa

- [REF-PEAK] D. Bianco, "Introducing the PEAK Threat Hunting Framework," Splunk SURGe, 04/2023. https://www.splunk.com/en_us/blog/security/peak-threat-hunting-framework.html
- D. Bianco, "Hypothesis-Driven Hunting with the PEAK Framework," Splunk SURGe, 05/2023. https://www.splunk.com/en_us/blog/security/peak-hypothesis-driven-threat-hunting.html
- D. Bianco, "Baseline Hunting with the PEAK Framework," Splunk SURGe, 07/2023. https://www.splunk.com/en_us/blog/security/peak-baseline-hunting.html
- R. Fetterman, "Model-Assisted Threat Hunting (M-ATH) with the PEAK Framework," Splunk SURGe, 05/2023. https://www.splunk.com/en_us/blog/security/peak-framework-math-model-assisted-threat-hunting.html
- PEAK content repo (Splunk SURGe, nay lưu tại Cisco-Talos). <https://github.com/splunk/peak>
- [REF-TAHITI] FI-ISAC, "TaHiTi: Targeted Hunting integrating Threat Intelligence." <https://www.nvb.nl/themas/veilig-bankieren/fi-isac/tahiti/>
- [REF-USENIX-TH] Maxam và cộng sự, "An Empirical Study of Threat Hunting," USENIX Security 2024. <https://www.usenix.org/conference/usenixsecurity24/presentation/maxam>

10.2. Điều tra đa chặng và đồ thị nguồn gốc

- [REF-SLEUTH] Hossain và cộng sự, "SLEUTH: Real-time Attack Scenario Reconstruction from COTS Audit Data," USENIX Security 2017. <https://www.usenix.org/system/files/conference/usenixsecurity17/sec17-hossain.pdf>
- [REF-HOLMES] Milajerdi và cộng sự, "HOLMES: Real-time APT Detection through Correlation of Suspicious Information Flows," IEEE S&P 2019. <https://ieeexplore.ieee.org/document/8835390/>
- [REF-OMEGALOG] Hassan và cộng sự, "OmegaLog: High-Fidelity Attack Investigation via Transparent Multi-layer Log Analysis," NDSS 2020. <https://experts.illinois.edu/en/publications/omegalog-high-fidelity-attack-investigation-via-transparent-multi/>
- [REF-DARPA-TC] DARPA Transparent Computing engagement data. <https://github.com/darpa-i2o/Transparent-Computing>

10.3. Truy vấn logic có kiểu và trích xuất hành vi

- [REF-AIQL] Gao và cộng sự, "AIQL: Enabling Efficient Attack Investigation from System Monitoring Data," USENIX ATC 2018. <https://www.usenix.org/system/files/conference/atc18/atc18-gao.pdf>

- [REF-THREATRAPTOR] Gao và cộng sự, "Enabling Efficient Cyber Threat Hunting with Cyber Threat Intelligence (ThreatRaptor)," ICDE 2021. <https://github.com/peng-gao-lab/threatraptor>

10.4. Vòng giả thuyết – bằng chứng – hành động

- [REF-ECTH] "Evidential Cyber Threat Hunting," arXiv:2104.10319. <https://arxiv.org/abs/2104.10319>
- [REF-ATHAFI] "ATHAFI: Agile Threat Hunting And Forensic Investigation," arXiv:2003.03663. <https://arxiv.org/abs/2003.03663>
- [REF-EXCYTIN] Microsoft Research, "ExCyTin-Bench: Evaluating LLM Agents on Cyber Threat Investigation." <https://www.microsoft.com/en-us/research/publication/excytin-bench-evaluating-llm-agents-on-cyber-threat-investigation/>

10.5. Schema, năng lực và chuẩn hóa

- [REF-OCSF] Open Cybersecurity Schema Framework. <https://ocsf.io/>
- [REF-MITRE-DC] MITRE ATT&CK Data Components. <https://attack.mitre.org/datacomponents/>
- [REF-MITRE-ANALYTICS] MITRE ATT&CK Detection Strategies. <https://attack.mitre.org/detectionstrategies/>
- [REF-OTEL] OpenTelemetry Semantic Conventions — Events. <https://opentelemetry.io/docs/specs/semconv/general/events/>
- [REF-MICROSOFT] Microsoft, "Advanced hunting with Security Copilot." <https://learn.microsoft.com/en-us/defender-xdr/advanced-hunting-security-copilot>

10.6. Khớp schema bằng mô hình ngôn ngữ, có kiểm soát

- [REF-SCHEMA-LLM] "Schema Matching with Large Language Models," arXiv:2407.11852. <https://arxiv.org/abs/2407.11852>
- [REF-REMATCH] "ReMatch: Retrieval Enhanced Schema Matching with LLMs," arXiv:2403.01567. <https://arxiv.org/abs/2403.01567>
- [REF-CHESS] "CHESS: Contextual Harnessing for Efficient SQL Synthesis," arXiv:2405.16755. <https://arxiv.org/abs/2405.16755>
- [REF-RAT-SQL] "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," arXiv:1911.04942. <https://arxiv.org/abs/1911.04942>
- [REF-ADAPTIVE-K] "Adaptive-K" (đã xem, không chọn làm chính sách chọn nguồn), arXiv:2506.08479. <https://arxiv.org/abs/2506.08479>
- [REF-TOOLSHED] "ToolShed," arXiv:2410.14594. <https://arxiv.org/abs/2410.14594>
- [REF-MDB-LINK] "MDB-Link," arXiv:2608.09588. <https://arxiv.org/abs/2608.09588>

10.7. Dùng công cụ an toàn và lập kế hoạch theo bằng chứng

- [REF-SAFE-TOOLS] "Verifiably Safe Tool Use for LLM Agents." <https://doi.org/10.1145/3786582.3786839>

- [REF-RPG] "Retrieve-Plan-Generation," EMNLP 2024. <https://aclanthology.org/2024.emnlp-main.270/>

10.8. Bộ dữ liệu đánh giá

- Splunk, "Boss of the SOC (BOTS) v1 Dataset." https://github.com/splunk/botsv1_data_set

Ghi chú về mức độ tin cậy nguồn

Theo 03_LITERATURE-AND-TRACEABILITY.md: các bài bình duyệt và chuẩn chính thức nâng đỡ *nguyên tắc*; các tiền ấn phẩm (arXiv) và tài liệu nhà cung cấp/thực hành nâng đỡ *thiết kế* với trọng số yếu hơn. Một lần chạy lại trong kho chỉ chứng minh đúng hành vi đã đo của lần đó, không khái quát ra mọi môi trường.

Cảnh báo kiểm chứng: một vài mã số arXiv trong danh mục (ví dụ REF-ADAPTIVE-K arXiv:2506.08479, REF-MDB-LINK arXiv:2608.09588) mang năm ở tương lai so với nội dung và được giữ nguyên đúng như bản gốc trong 03_LITERATURE-AND-TRACEABILITY.md. Nếu dùng báo cáo này cho mục đích xuất bản chính thức, cần kiểm chứng lại các mã số và liên kết trước khi nộp.

Chương 11 — Phụ lục

Phụ lục A. Ánh xạ thuật ngữ

Bảng 11.1.

Tiếng trong mã	Nghĩa dùng trong báo cáo
Hunt	Một lần săn có yêu cầu, số cái và tài khoản cuối
Claim	Mệnh đề cần chứng minh, thuộc đồ thị tương thích
Goal	Quan hệ hoặc câu trả lời trong SemanticGoalGraph
Capability	Thao tác nhà cung cấp đã khai báo hoặc đã probe
Observation	Một hàng telemetry đã nhập số cái
Evidence card	Nhóm quan sát đã nén để đánh giá và báo cáo
Cell (ô phủ)	(ProviderScope, thực thể hoặc ANY, bucket thời gian)
Probe	Truy vấn nhỏ kiểm một ánh xạ nguồn, không kiểm sự cố
Binding	Giá trị một biến nhận được, kèm nguồn gốc
Candidate	Giá trị/năng lực được phép dùng để truy hồi, chưa dùng để kết luận
Complete	Lần truy vấn tới điểm hoàn tất mà nhà cung cấp khai báo
Route exhausted	Mọi cách có giới hạn của một quan hệ đã thử hoặc bị từ chối có lý do
Testimony	Lời người, không phải telemetry
Disposition	Nhãn cuối của vòng cảnh báo
Stopping decision	Lý do máy dừng
Act	Bản nháp phát hiện, backlog và tóm tắt sau lần săn

Phụ lục B. Các quyết định dừng (StoppingDecision)

Bảng 11.2.

Giá trị	Khi nào mã đặt
STOP_RESOLVED	Tuyến ngữ nghĩa đã chứng minh đủ để trả lời
STOP_REFUTED	Bằng chứng bác giả thuyết theo luật đã khai báo
STOP_INCONCLUSIVE_IDENTITY_UNRESOLVED	Chưa nối được định danh cần để trả lời
STOP_INCONCLUSIVE_RELATION_UNPROVEN	Còn quan hệ chưa chứng minh
STOP_INCONCLUSIVE_COVERAGE_GAP	Truy vấn dở dang hoặc nguồn không phủ
STOP_NEEDS_USER_DECISION	Nhiều ràng buộc, cần người thu hẹp
STOP_EXHAUSTED_BY_BUDGET	Hết lượt, truy vấn, token hoặc thời gian
STOP_UNSUPPORTED_CAPABILITY	Có nhà cung cấp sống nhưng không năng lực cho claim
STOP_UNREACHABLE	Không nhà cung cấp nào trực tuyến
STOP_INSUFFICIENT	Giả thuyết hoặc mô hình không đủ đặc tả

Phụ lục C. Tài liệu nội bộ liên quan

Bảng 11.3.

Văn bản	Vai trò
01_FINAL-ARCHITECTURE.md	Kiến trúc chuẩn v7
02_METHOD-AND-IMPLEMENTATION-PLAN.md	Phương pháp và kế hoạch mã
03_LITERATURE-AND-TRACEABILITY.md	Nguồn ngoài và biên giới tuyên bố
04-IMPLEMENTATION-CHECKLIST.md	Cổng bằng chứng
docs/PEAK-PREPARE-GATE.md	Cổng PEAK Prepare bắt buộc (đóng góp mới)
docs/PEAK-RESEARCH-AND-MAPPING.md	Nghiên cứu PEAK và đối chiếu trung thực
docs/EVAL-GROUND-TRUTH.md	Đánh giá trên BOTS v1 thật
docs/BOTS-V1-WALKTHROUGH.md	Hướng dẫn chạy trên BOTS v1
docs/BAO-CAO-LLM-JUDGE.md	Báo cáo LLM judge
docs/PEAK-END-TO-END-DEMO.md	Demo baseline → math → PoC trên sample 16 sự kiện

Phụ lục D. Danh mục đầy đủ lớp và hàm

Danh mục đầy đủ **260 lớp và 658 hàm/phương thức** của src/hunting, kèm khoảng dòng và câu đầu docstring lấy trực tiếp từ mã, nằm ở **Phụ lục A của bản một-tệp** docs/PAPER-BAO-CAO-HE-THONG.md. Bản nhiều chương này không lặp lại danh mục đó để giữ mỗi chương tập trung vào một nội dung; hai bản trở tới cùng một mã nguồn.

Phụ lục E. Lệnh tái chạy đánh giá

1. Nạp eval DB (cần data/raw/*.gz, ~6 phút)

```
.venv\Scripts\python.exe scripts/ingest_botsv1_eval.py
```

```
.venv\Scripts\python.exe scripts/ingest_http.py
```

2. Độ đặc hiệu: 4 PoC trên 4.38M dòng (~30s)

```
.venv\Scripts\python.exe scripts/run_fp_eval.py
```

3. Độ nhạy: Joomla RCE + judge

```
.venv\Scripts\python.exe main.py --provider cdb --db data/botsv1_eval.sqlite `
    --poc-file pocs/poc-joomla-rce.json `
    --time-window "2016-08-10T21:36:00Z/2016-08-10T22:00:00Z" `
    --poc-judge --llm api --poc-judge-max-tokens 16000
```

4. Bộ kiểm thử phần mềm

```
.venv\Scripts\python.exe -m pytest tests/unit -q
```

